Programme : Bachelor of Computer Science (*Data Engineering*)

Subject Code : SECP2753

Subject Name : Data Mining

Session-Sem : 2024/2025-2

Group Members:

Name	Matric Number
LAU YAN KAI	**A23CS0098**
CHEW CHIU XIAN	**A23CS0061**
NG YU HIN	**A23CS0148**

Table of Contents

Introduction

This project uses a real-life dataset from a survey about mental health in the tech industry. The survey has over 1,200 answers from people who work in technology jobs around the world. It includes about 25 pieces of information about each person, like their job, mental health history, and how their workplace supports them.

Even though the data comes from a survey and not a doctor, it still gives us a good idea of the mental health problems that tech workers may face. This dataset is safe to use and is helpful for learning, studying patterns, and building models that can make predictions. In this project, we will use Python to clean the data, prepare it, build models, and check how well the models work. We will use tools like Pandas, Scikit-learn, and Matplotlib to help with this.

We will use three machine learning models including Logistic Regression, Decision Tree, and Random Forest. These models will help us predict things like whether someone will get help for their mental health or whether their workplace supports them. We will test how good these models are using scores like accuracy, precision, recall, F1-score, and a confusion matrix. The main goal of this project is to practice using data to learn useful things and to show how data can help make workplaces better for mental health.

Business and Data Understanding

Problem Statement

In recent years, mental health has become a key concern in many industries, particularly within the technology sector, where fast-paced environments, tight deadlines, and long working hours are common. Despite growing awareness, many organizations still struggle to detect early signs of mental health issues among employees, often due to lack of data-driven tools or fear of stigma within the workplace.

This gap highlights the importance of developing intelligent systems that can help identify individuals who may require support. Unlike traditional approaches, which rely on general observation or reactive policies, machine learning offers a proactive method to recognize patterns in employee behaviour, experiences, and perceptions related to mental health.

In this project, we aim to build predictive models that can classify whether an employee is likely to seek mental health treatment, whether they find it easy to take mental health leave, and whether their employer provides adequate mental health benefits. These targets are closely tied to key workplace factors such as openness, support systems, and company culture.

To achieve this, we apply three supervised learning algorithms Logistic Regression, Decision Tree, and Random Forest are chosen for their interpretability, flexibility, and ability to handle a wide range of data types. The overall objective is not only to improve prediction accuracy but also to explore how data mining can support early intervention, reduce mental health stigma, and encourage healthier working environments.

Dataset Overview

The dataset used in this project comes from the Mental Health in Tech Survey. It includes answers from more than 1,250 people who work in technology jobs. The answers were collected through an online survey, and no names or private information were included, so the data is safe and anonymous. Many researchers use this dataset to study how mental health affects people at work.

The dataset has about 25 pieces of information (called features) for each person. These include things like age, gender, where they work, whether they work remotely, and how they feel about mental health in their workplace. It also includes information about whether their company supports mental health and if they have had mental health problems before.

In this project, we focus on three main target variables we want to predict:

1. `care_options` – Whether the person knows what mental health care options their employer offers.
2. `treatment` – Whether the person has ever asked for help or treatment for a mental health issue.
3. `family_history` – Whether the person has family members who have had mental health problems.

These targets help us learn more about how mental health is treated and supported in different workplaces, and how people's background and experiences affect their choices.

Attribute Summary

Attribute Name	Description	Type
`Age`	Respondent's age	Numerical
`Gender`	Self-identified gender	Categorical
`Country`	Country of residence	Categorical
`state`	State	Categorical

Attribute Name	Description	Type
self_employed	Whether self-employed	Categorical
family_history	Family history of mental illness	Categorical
treatment	Has sought mental health treatment	Categorical
work_interfere	Frequency of mental health interfering with work	Categorical
no_employees	Company size	Categorical
remote_work	Works remotely	Categorical
tech_company	Works in a tech company	Categorical
benefits	Employer provides mental health benefits	Categorical
care_options	Employer offers mental health care options	Categorical
wellness_program	Employer has wellness programs	Categorical
seek_help	Ease of seeking help from employer	Categorical
anonymity	Is anonymity protected at work	Categorical
leave	Ease of taking medical leave for mental health	Categorical
mental_health_consequence	Fear of consequences if mental health is discussed	Categorical
phys_health_consequence	Fear of consequences if physical health is discussed	Categorical
coworkers	Comfort discussing mental health with co-workers	Categorical
supervisor	Comfort discussing mental health with supervisor	Categorical
mental_health_interview	Comfort discussing mental health in job interviews	Categorical
phys_health_interview	Comfort discussing physical health in job interviews	Categorical
mental_vs_physical	Perceived importance of mental vs physical health	Categorical
obs_consequence	Observed negative consequences of mental health disclosure	Categorical

This dataset has many useful details that help us study mental health and support at work. It also allows us to predict more than one thing at a time, like if someone will ask for help or if their company offers support. In the next part, we will explain how we cleaned and prepared the data so that we can use it to train machine learning models and make predictions.

Classification Models Selected for Analysis

In this project, we use supervised learning to build classification models. These models help us predict specific workplace mental health outcomes, such as whether an employee receives mental health benefits, finds it easy to take leave, or struggles with mental health interference at work. We selected the following three popular classification models for analysis:

1. *Logistic Regression*

Logistic Regression is a simple model that is good for answering yes-or-no questions. It looks at the data and tries to find patterns to tell us if something is likely to happen. In our project, we use it to guess things like whether someone's work is affected by mental health or if their company gives mental health benefits.

2. *Decision Tree*

A Decision Tree is a flowchart-like structure that splits the dataset into branches based on decision rules. It is highly interpretable and allows us to understand which features such as age, remote work status, or anonymity policies play a key role in influencing the target outcomes. We use this model to explore how various workplace factors impact employee mental health experiences.

3. *Random Forest*

Random Forest is like using many decision trees at once. It builds lots of small trees and combines their answers to make better guesses. This model helps give more accurate results, especially when the data is big or has many different types.

These three models were chosen because they are easy to use, work well with this kind of data, and are helpful for both school projects and real-life research. We will train each model, test it, and compare how well they work using scores like accuracy and confusion matrix.

Data Preparation and Preprocessing

Checking for Missing Values

```
# Check missing values
missing_values = df.isnull().sum()
missing_values = missing_values[missing_values > 0]

# Display missing values as a table
missing_table = pd.DataFrame({
    'Missing Values': missing_values.values,
    'Percentage Missing (%)': (missing_values / len(df) *
100).round(2)
})
print(missing_table)
```

```
                Missing Values  Percentage Missing (%)
state                      515                   40.91
self_employed               18                    1.43
work_interfere             264                   20.97
comments                  1095                   86.97
```

Data Cleaning

Remove and select only the relevant features for our study.

```
df.drop(columns=['Timestamp', 'state', 'comments', 'Country'], inplace
= True)
```

Check if there is outlier for Age and Gender

```
print(df['Age'].unique())
```

```
[         37          44          32          31          33          35
          39          42          23          29          36          27
          46          41          34          30          40          38
          50          24          18          28          26          22
          19          25          45          21         -29          43
          56          60          54         329          55 99999999999
          48          20          57          58          47          62
          51          65          49       -1726           5          53
          61           8          11          -1          72]
```

```
print(df['Gender'].unique())
```

```
['Female' 'M' 'Male' 'male' 'female' 'm' 'Male-ish' 'maile' 'Trans-female'
 'Cis Female' 'F' 'something kinda male?' 'Cis Male' 'Woman' 'f' 'Mal'
 'Male (CIS)' 'queer/she/they' 'non-binary' 'Femake' 'woman' 'Make' 'Nah'
 'All' 'Enby' 'fluid' 'Genderqueer' 'Female ' 'Androgyne' 'Agender'
 'cis-female/femme' 'Guy (-ish) ^_^' 'male leaning androgynous' 'Male '
 'Man' 'Trans woman' 'msle' 'Neuter' 'Female (trans)' 'queer'
 'Female (cis)' 'Mail' 'cis male' 'A little about you' 'Malr' 'p' 'femail'
 'Cis Man' 'ostensibly male, unsure what that really means']
```

Clean the data for Age variable by only accept age from 1 to 100

```
df.drop(df[df['Age'] < 0].index, inplace = True)
df.drop(df[df['Age'] > 100].index, inplace = True)
df['Age'].unique()
```

```
array([37, 44, 32, 31, 33, 35, 39, 42, 23, 29, 36, 27, 46, 41, 34, 30, 40,
       38, 50, 24, 18, 28, 26, 22, 19, 25, 45, 21, 43, 56, 60, 54, 55, 48,
       20, 57, 58, 47, 62, 51, 65, 49,  5, 53, 61,  8, 11, 72])
```

Collect and group all non-binary, unclear, or miscellaneous entries of Gender into 3 standard categories into a general category.

```
df['Gender'].replace(['Male ', 'male', 'M', 'm', 'Male', 'Cis Male',
                      'Man', 'cis male', 'Mail', 'Male-ish', 'Male (CIS)',
                       'Cis Man', 'msle', 'Malr', 'Mal', 'maile', 'Make',],
'Male', inplace = True)

df['Gender'].replace(['Female ', 'female', 'F', 'f', 'Woman', 'Female',
                      'femail', 'Cis Female', 'cis-female/femme', 'Femake',
'Female (cis)',
                      'woman',], 'Female', inplace = True)

df["Gender"].replace(['Female (trans)', 'queer/she/they', 'non-binary',
                      'fluid', 'queer', 'Androgyne', 'Trans-female', 'male
leaning androgynous',
                       'Agender', 'A little about you', 'Nah', 'All',
                       'ostensibly male, unsure what that really means',
                       'Genderqueer', 'Enby', 'p', 'Neuter', 'something
kinda male?',
                       'Guy (-ish) ^_^', 'Trans woman',], 'Other', inplace =
True)

df['Gender'].value_counts()
```

Gender	count
Male	988
Female	247
Other	19

dtype: int64

Check again if there is outlier for Age and Gender

```
print(df['Age'].unique())
```

```
[37 44 32 31 33 35 39 42 23 29 36 27 46 41 34 30 40 38 50 24 18 28 26 22
 19 25 45 21 43 56 60 54 55 48 20 57 58 47 62 51 65 49  5 53 61  8 11 72]
```

```
print(df['Gender'].unique())
```

```
['Female' 'Male' 'Other']
```

Data Preparation

Change the NaN value in work_interfere to ‘Unsure’

```
df['work_interfere'] = df['work_interfere'].fillna('Unsure' )
print(df['work_interfere'].unique())
```

```
['Often' 'Rarely' 'Never' 'Sometimes' 'Unsure']
```

Change the NaN value in self_employment to ‘No’

```
df['self_employed'] = df['self_employed'].fillna('No')
print(df['self_employed'].unique())
```

```
['No' 'Yes']
```

Encoding Categorical Variables

```
input = ['Gender', 'self_employed', 'family_history', 'treatment',
       'work_interfere', 'no_employees', 'remote_work', 'tech_company',
       'benefits', 'care_options', 'wellness_program', 'seek_help',
       'anonymity', 'leave', 'mental_health_consequence',
       'phys_health_consequence', 'coworkers', 'supervisor',
       'mental_health_interview', 'phys_health_interview',
       'mental_vs_physical', 'obs_consequence']
label_encoder = LabelEncoder()

for col in input:
    label_encoder.fit(df[col])
    df[col] = label_encoder.transform(df[col])
```

	Age	Gender	self_employed	family_history	treatment	work_interfere	no_employees	remote_work	tech_company	benefits	...	anonymity	leave	mental_health_consequence	phys_healt
0	37	0	0	0	1	1	4	0	1	2	...	2	2	1	
1	44	1	0	0	0	2	5	0	0	0	...	0	0	0	
2	32	1	0	0	0	2	4	0	1	1	...	0	1	1	
3	31	1	0	1	1	1	2	0	1	1	...	1	1	2	
4	31	1	0	0	0	0	1	1	1	2	...	0	0	1	

5 rows × 23 columns

Scaling Age

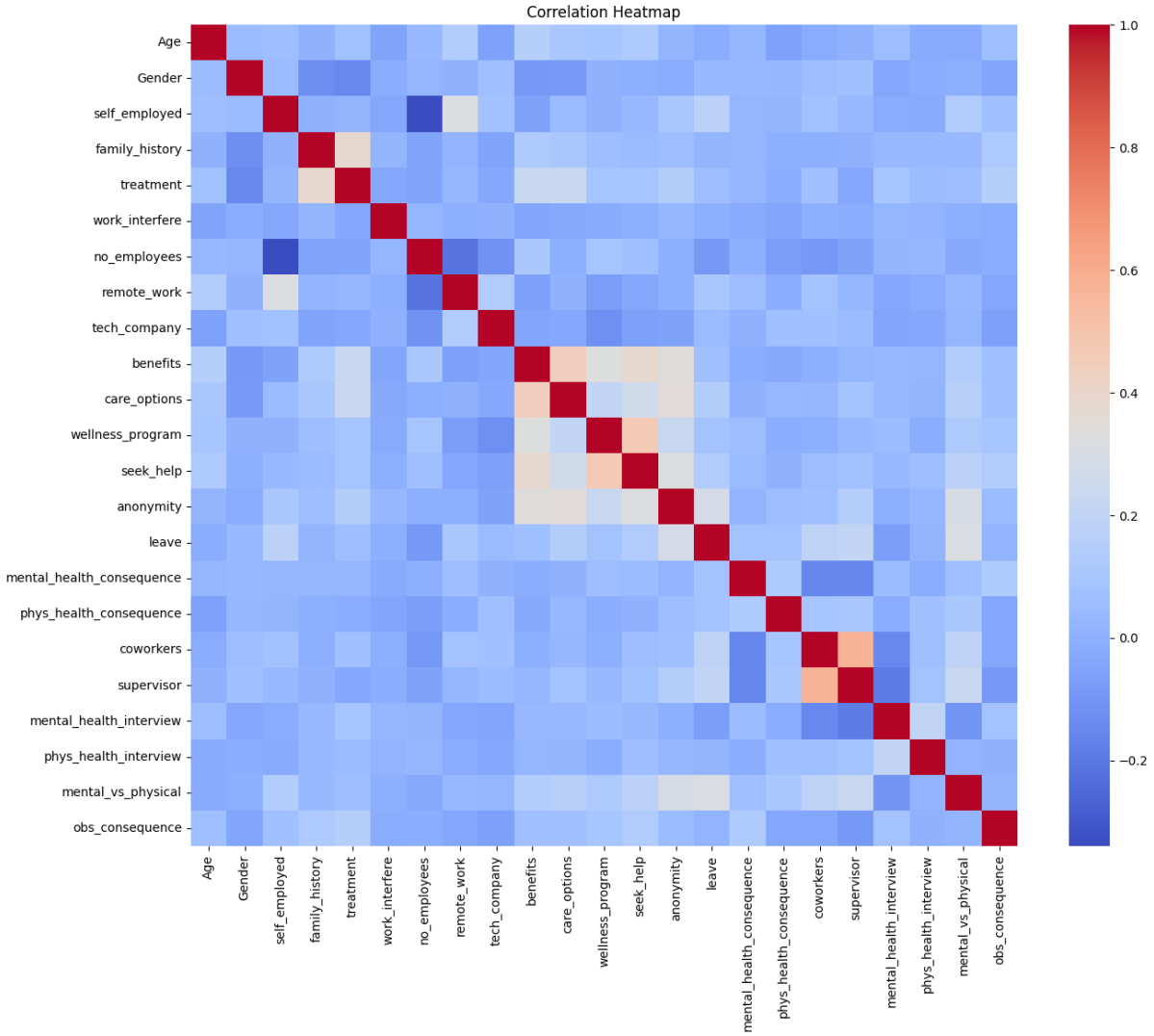
```
scaler = MinMaxScaler()
df['Age'] = scaler.fit_transform(df[['Age']])
df.head()
```

	Age	Gender	self_employed	family_history	treatment	work_interfere	no_employees	remote_work	tech_company	benefits	...	anonymity	leave	mental_health_consequence	phys_
0	0.477612	0	0	0	1	1	4	0	1	2	...	2	2	1	
1	0.582090	1	0	0	0	2	5	0	0	0	...	0	0	0	
2	0.402985	1	0	0	0	2	4	0	1	1	...	0	1	1	
3	0.388060	1	0	1	1	1	2	0	1	1	...	1	1	2	
4	0.388060	1	0	0	0	0	1	1	1	2	...	0	0	1	

5 rows × 23 columns

Correlation Matrix and Heatmap

```
corr_matrix = df.corr(numeric_only=True)
plt.figure(figsize=(15, 12))
sns.heatmap(corr_matrix, cmap="coolwarm", annot=False)
plt.title("Correlation Heatmap")
plt.show()
```

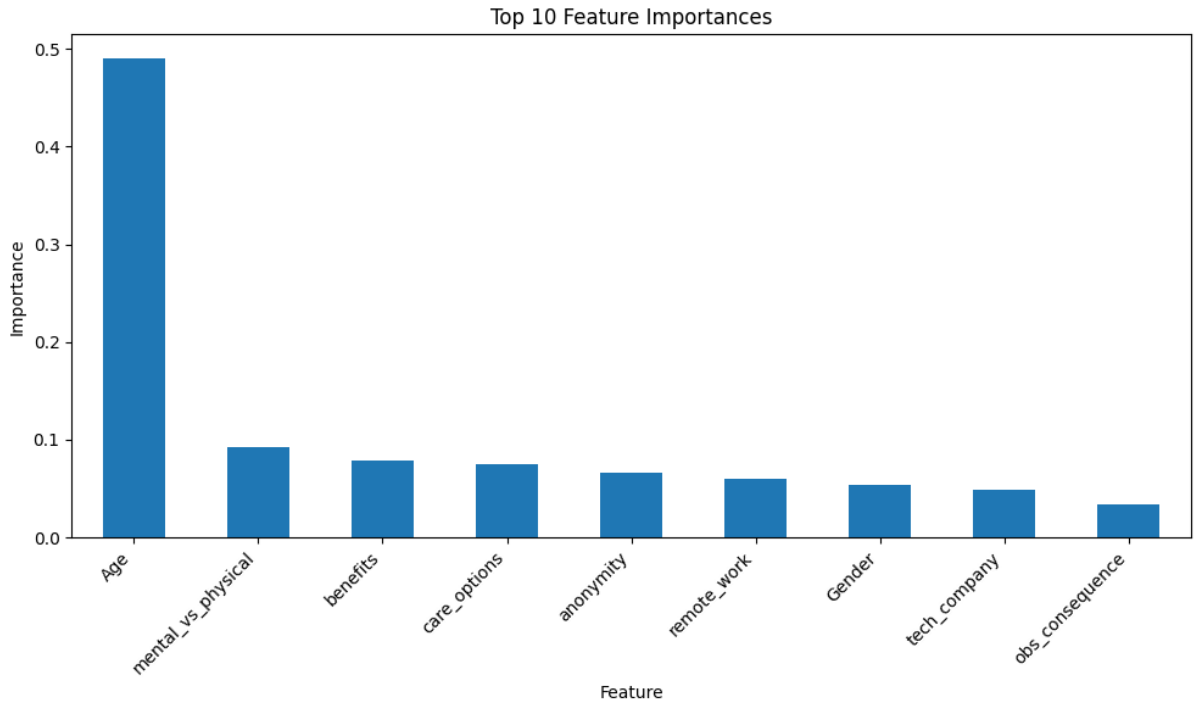


Model Development

```
feature_cols = ['Age', 'Gender', 'remote_work', 'tech_company',
'benefits', 'care_options', 'anonymity', 'mental_vs_physical',
'obs_consequence']
input = df[feature_cols]
target = df.work_interfere

X_train, X_test, y_train, y_test = train_test_split(input, target,
test_size=0.30, random_state=42)
```

Feature selection

```
model = RandomForestClassifier(n_estimators=100, random_state=0)
model.fit(X_train, y_train)

feature_importances = pd.Series(model.feature_importances_,
index=input.columns)

top_10_features = feature_importances.nlargest(10)

plt.figure(figsize=(10, 6))
top_10_features.plot(kind='bar')
plt.title('Top 10 Feature Importances')
plt.ylabel('Importance')
plt.xlabel('Feature')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

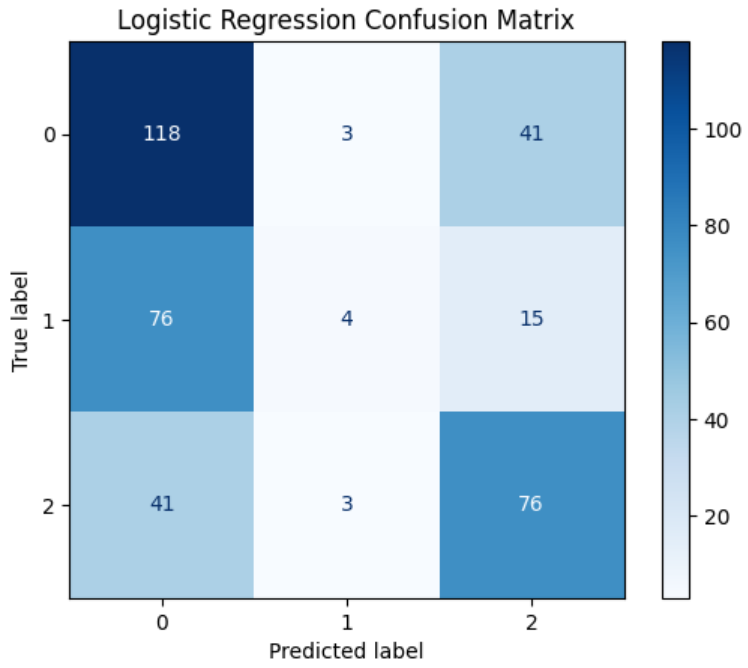


care_options

Logistic Regression

```
logreg = LogisticRegression()
logreg.fit(X_train, y_train)

y_pred_logreg = logreg.predict(X_test)

# Evaluate the model
accuracy_logreg = accuracy_score(y_test, y_pred_logreg)
precision_logreg = precision_score(y_test, y_pred_logreg,
average='weighted')
f1_logreg = f1_score(y_test, y_pred_logreg, average='weighted')

print(f"Logistic Regression Accuracy: {accuracy_logreg:.4f}")
print(f"Logistic Regression Precision: {precision_logreg:.4f}")
print(f"Logistic Regression F1-score: {f1_logreg:.4f}")

# Confusion Matrix
cm_logreg = confusion_matrix(y_test, y_pred_logreg)
disp_logreg = ConfusionMatrixDisplay(confusion_matrix=cm_logreg,
display_labels=logreg.classes_)
disp_logreg.plot(cmap=plt.cm.Blues)
plt.title('Logistic Regression Confusion Matrix')
plt.show()
```

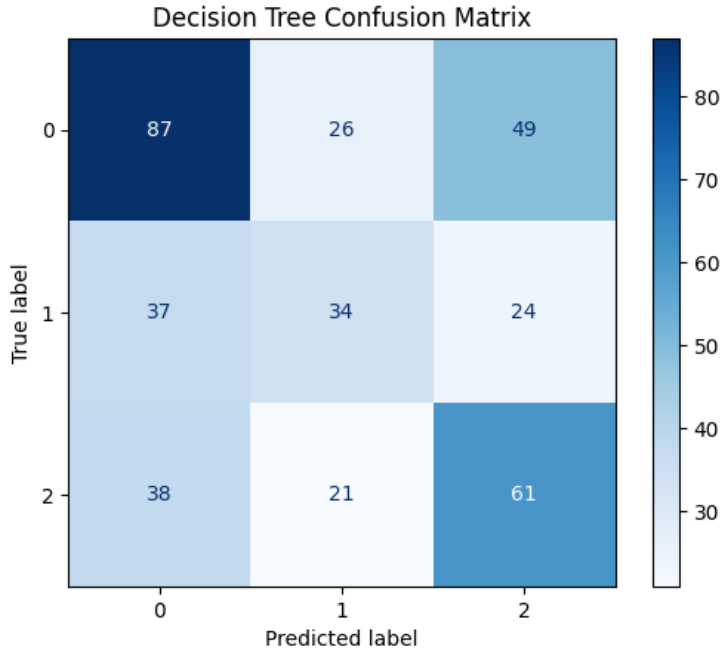
```
====== Logistic Regression ======
 Accuracy: 0.5252
 Precision: 0.4998
 F1-score: 0.4666
 Recall: 0.5252
```



Decision Tree

```
dtree = DecisionTreeClassifier()
dtree.fit(X_train, y_train)

y_pred_dtree = dtree.predict(X_test)

# Evaluate the model
accuracy_dtree = accuracy_score(y_test, y_pred_dtree)
precision_dtree = precision_score(y_test, y_pred_dtree,
average='weighted')
f1_dtree = f1_score(y_test, y_pred_dtree, average='weighted')
recall_dtree = recall_score(y_test, y_pred_dtree, average='weighted')

print('====== Decision Tree ======')
print(f" Accuracy: {accuracy_dtree:.4f}")
print(f" Precision: {precision_dtree:.4f}")
print(f" F1-score: {f1_dtree:.4f}")
print(f" Recall: {recall_dtree:.4f}")

# Confusion Matrix
cm_dtree = confusion_matrix(y_test, y_pred_dtree)
disp_dtree = ConfusionMatrixDisplay(confusion_matrix=cm_dtree,
display_labels=dtree.classes_)
disp_dtree.plot(cmap=plt.cm.Blues)
plt.title('Decision Tree Confusion Matrix')
plt.show()
```

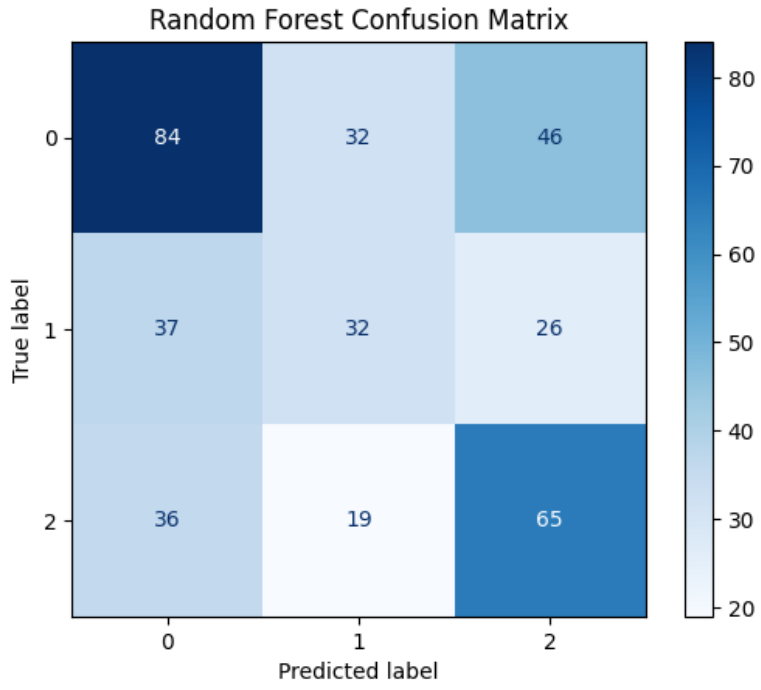
```
====== Decision Tree ======
 Accuracy: 0.4828
 Precision: 0.4814
 F1-score: 0.4810
 Recall: 0.4828
```



Random Forest

```
rf_model = RandomForestClassifier(n_estimators=100, random_state=0)
rf_model.fit(X_train, y_train)

y_pred_rf = rf_model.predict(X_test)

# Evaluate the model
accuracy_rf = accuracy_score(y_test, y_pred_rf)
precision_rf = precision_score(y_test, y_pred_rf, average='weighted')
f1_rf = f1_score(y_test, y_pred_rf, average='weighted')
recall_rf = recall_score(y_test, y_pred_rf, average='weighted')

print('====== Random Forest ======')
print(f" Accuracy: {accuracy_rf:.4f}")
print(f" Precision: {precision_rf:.4f}")
print(f" Recall: {recall_rf:.4f}")
print(f" F1-score: {f1_rf:.4f}")

# Confusion Matrix
cm_rf = confusion_matrix(y_test, y_pred_rf)
disp_rf = ConfusionMatrixDisplay(confusion_matrix=cm_rf,
display_labels=rf_model.classes_)
disp_rf.plot(cmap=plt.cm.Blues)
plt.title('Random Forest Confusion Matrix')
plt.show()
```

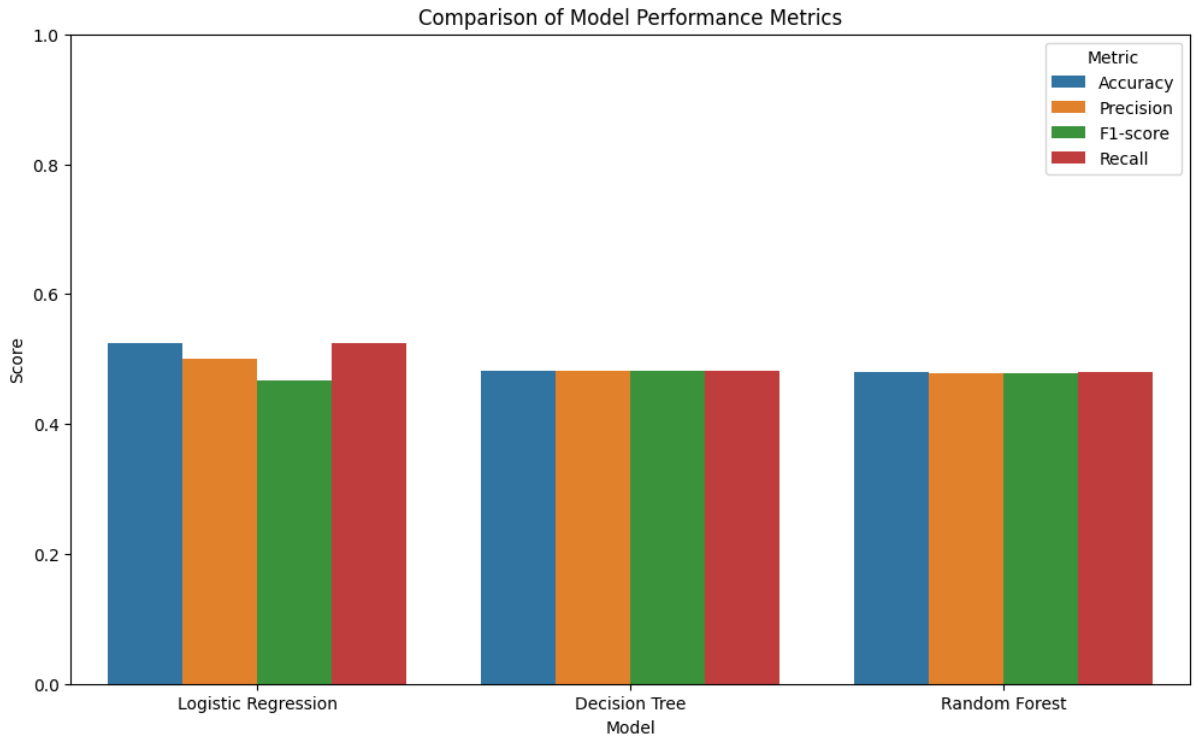
```
====== Random Forest ======
 Accuracy: 0.4801
 Precision: 0.4781
 Recall: 0.4801
 F1-score: 0.4779
```



Comparison between Models

```
results_df = pd.DataFrame({
    'Model': ['Logistic Regression', 'Decision Tree', 'Random Forest'],
    'Accuracy': [accuracy_logreg, accuracy_dtree, accuracy_rf],
    'Precision': [precision_logreg, precision_dtree, precision_rf],
    'F1-score': [f1_logreg, f1_dtree, f1_rf],
    'Recall': [recall_logreg, recall_dtree, recall_rf]
})

print("Model Comparison Table:")
print(results_df.to_string(index=False))

results_melted = results_df.melt('Model', var_name='Metric',
value_name='Score')
plt.figure(figsize=(12, 7))
sns.barplot(x='Model', y='Score', hue='Metric', data=results_melted)
plt.title('Comparison of Model Performance Metrics')
plt.ylabel('Score')
plt.ylim(0, 1)
plt.show()
```

```
              Model  Accuracy  Precision  F1-score   Recall
Logistic Regression  0.525199   0.499829  0.466635 0.525199
      Decision Tree  0.482759   0.481441  0.481014 0.482759
      Random Forest  0.480106   0.478080  0.477916 0.480106
```



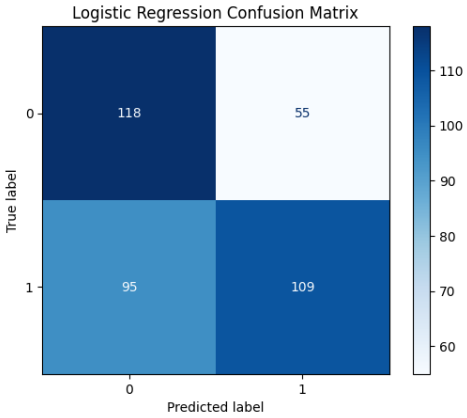
treatment

```
target = df.treatment
X_train, X_test, y_train, y_test = train_test_split(input, target,
test_size=0.30, random_state=42)
```

Logistic Regression

```
logreg = LogisticRegression()
logreg.fit(X_train, y_train)

y_pred_logreg = logreg.predict(X_test)

# Evaluate the model
accuracy_logreg = accuracy_score(y_test, y_pred_logreg)
precision_logreg = precision_score(y_test, y_pred_logreg,
average='weighted')
f1_logreg = f1_score(y_test, y_pred_logreg, average='weighted')
recall_logreg = recall_score(y_test, y_pred_logreg, average='weighted')

print('====== Logistic Regression ======')
print(f" Accuracy: {accuracy_logreg:.4f}")
print(f" Precision: {precision_logreg:.4f}")
print(f" F1-score: {f1_logreg:.4f}")
print(f" Recall: {recall_logreg:.4f}")

# Confusion Matrix
cm_logreg = confusion_matrix(y_test, y_pred_logreg)
disp_logreg = ConfusionMatrixDisplay(confusion_matrix=cm_logreg,
display_labels=logreg.classes_)
disp_logreg.plot(cmap=plt.cm.Blues)
plt.title('Logistic Regression Confusion Matrix')
plt.show()
```

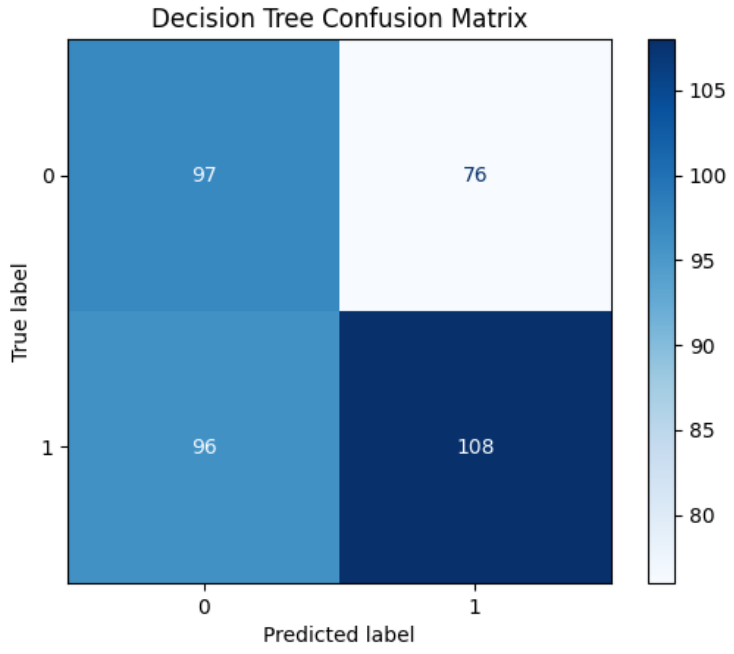
```
====== Logistic Regression ======
 Accuracy: 0.6021
 Precision: 0.6139
 F1-score: 0.6011
 Recall: 0.6021
```



Decision Tree

```
dtree = DecisionTreeClassifier()
dtree.fit(X_train, y_train)

y_pred_dtree = dtree.predict(X_test)

# Evaluate the model
accuracy_dtree = accuracy_score(y_test, y_pred_dtree)
precision_dtree = precision_score(y_test, y_pred_dtree,
average='weighted')
f1_dtree = f1_score(y_test, y_pred_dtree, average='weighted')
recall_dtree = recall_score(y_test, y_pred_dtree, average='weighted')

print('====== Decision Tree ======')
print(f" Accuracy: {accuracy_dtree:.4f}")
print(f" Precision: {precision_dtree:.4f}")
print(f" F1-score: {f1_dtree:.4f}")
print(f" Recall: {recall_dtree:.4f}")

# Confusion Matrix
cm_dtree = confusion_matrix(y_test, y_pred_dtree)
disp_dtree = ConfusionMatrixDisplay(confusion_matrix=cm_dtree,
display_labels=dtree.classes_)
disp_dtree.plot(cmap=plt.cm.Blues)
plt.title('Decision Tree Confusion Matrix')
plt.show()
```

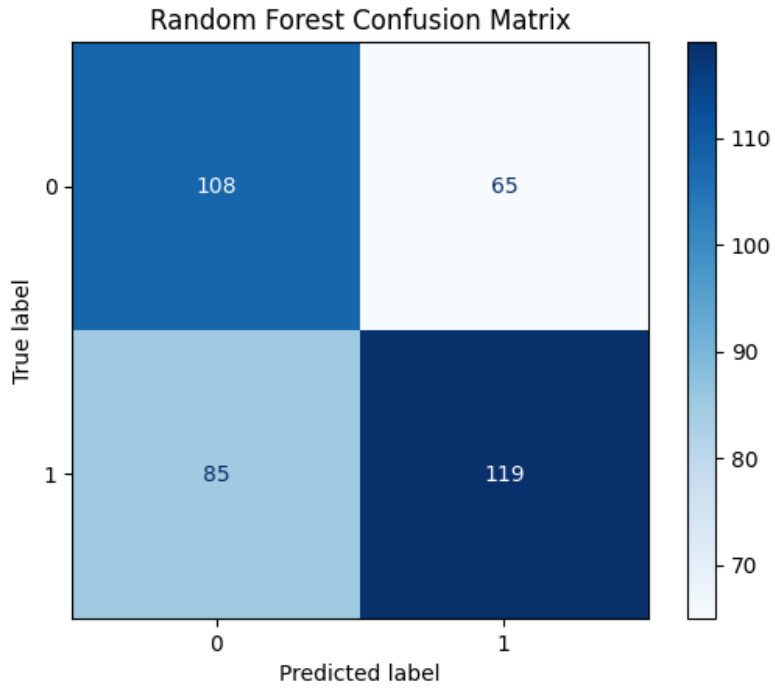
```
====== Decision Tree ======
 Accuracy: 0.5438
 Precision: 0.5482
 F1-score: 0.5445
 Recall: 0.5438
```



Random Forest

```
rf_model = RandomForestClassifier(n_estimators=100, random_state=0)
rf_model.fit(X_train, y_train)

y_pred_rf = rf_model.predict(X_test)

# Evaluate the model
accuracy_rf = accuracy_score(y_test, y_pred_rf)
precision_rf = precision_score(y_test, y_pred_rf, average='weighted')
f1_rf = f1_score(y_test, y_pred_rf, average='weighted')
recall_rf = recall_score(y_test, y_pred_rf, average='weighted')

print('====== Random Forest ======')
print(f" Accuracy: {accuracy_rf:.4f}")
print(f" Precision: {precision_rf:.4f}")
print(f" Recall: {recall_rf:.4f}")
print(f" F1-score: {f1_rf:.4f}")

# Confusion Matrix
cm_rf = confusion_matrix(y_test, y_pred_rf)
disp_rf = ConfusionMatrixDisplay(confusion_matrix=cm_rf,
display_labels=rf_model.classes_)
disp_rf.plot(cmap=plt.cm.Blues)
plt.title('Random Forest Confusion Matrix')
plt.show()
```

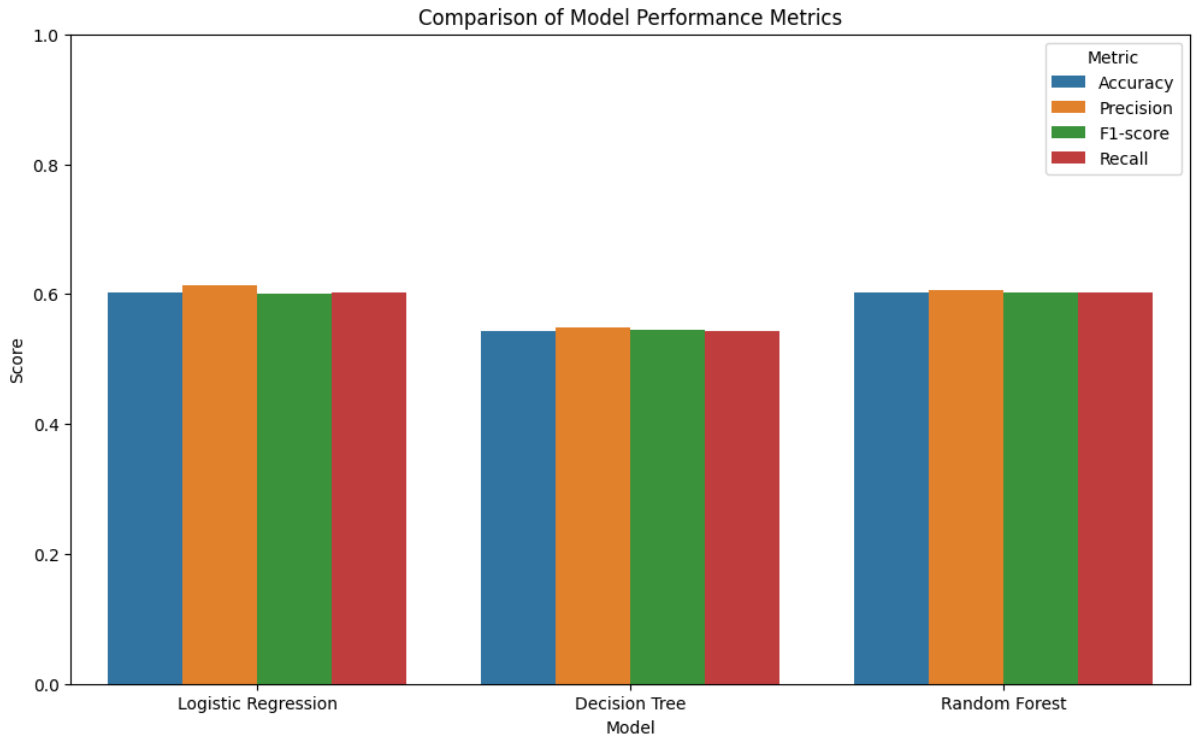
```
====== Random Forest ======
 Accuracy: 0.6021
 Precision: 0.6067
 Recall: 0.6021
 F1-score: 0.6027
```



Comparison between Models

```
results_df = pd.DataFrame({
    'Model': ['Logistic Regression', 'Decision Tree', 'Random Forest'],
    'Accuracy': [accuracy_logreg, accuracy_dtree, accuracy_rf],
    'Precision': [precision_logreg, precision_dtree, precision_rf],
    'F1-score': [f1_logreg, f1_dtree, f1_rf],
    'Recall': [recall_logreg, recall_dtree, recall_rf]
})

print(results_df.to_string(index=False))

results_melted = results_df.melt('Model', var_name='Metric',
value_name='Score')
plt.figure(figsize=(12, 7))
sns.barplot(x='Model', y='Score', hue='Metric', data=results_melted)
plt.title('Comparison of Model Performance Metrics')
plt.ylabel('Score')
plt.ylim(0, 1)
plt.show()
```

```
              Model  Accuracy  Precision  F1-score   Recall
Logistic Regression  0.602122   0.613861  0.601114 0.602122
      Decision Tree  0.543767   0.548242  0.544473 0.543767
      Random Forest  0.602122   0.606746  0.602738 0.602122
```



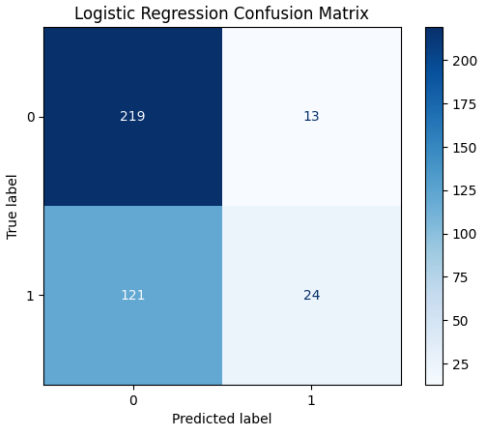
family_history

```
target = df.family_history
X_train, X_test, y_train, y_test = train_test_split(input, target,
test_size=0.30, random_state=42)
```

Logistic Regression

```
logreg = LogisticRegression()
logreg.fit(X_train, y_train)

y_pred_logreg = logreg.predict(X_test)

# Evaluate the model
accuracy_logreg = accuracy_score(y_test, y_pred_logreg)
precision_logreg = precision_score(y_test, y_pred_logreg,
average='weighted')
f1_logreg = f1_score(y_test, y_pred_logreg, average='weighted')
recall_logreg = recall_score(y_test, y_pred_logreg, average='weighted')

print('====== Logistic Regression ======')
print(f" Accuracy: {accuracy_logreg:.4f}")
print(f" Precision: {precision_logreg:.4f}")
print(f" F1-score: {f1_logreg:.4f}")
print(f" Recall: {recall_logreg:.4f}")

# Confusion Matrix
cm_logreg = confusion_matrix(y_test, y_pred_logreg)
disp_logreg = ConfusionMatrixDisplay(confusion_matrix=cm_logreg,
display_labels=logreg.classes_)
disp_logreg.plot(cmap=plt.cm.Blues)
plt.title('Logistic Regression Confusion Matrix')
plt.show()
```

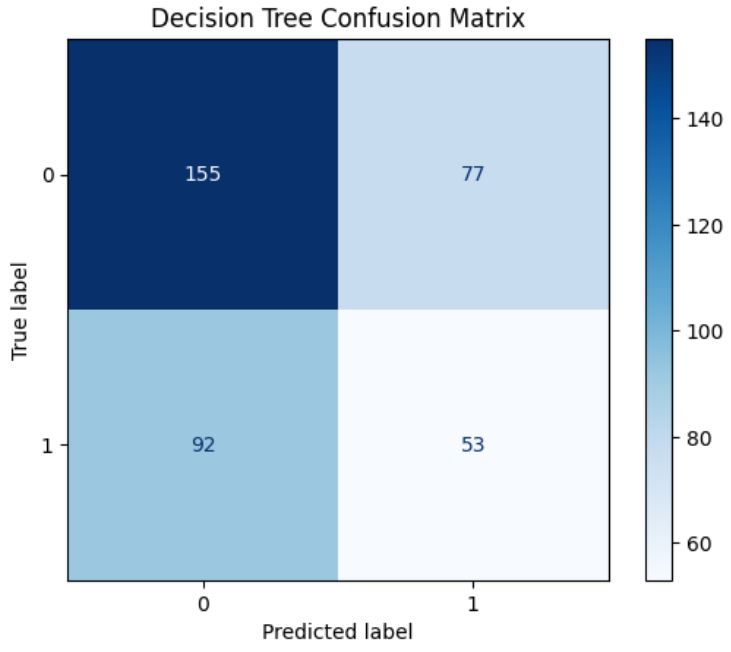
```
====== Logistic Regression ======
 Accuracy: 0.6446
 Precision: 0.6459
 F1-score: 0.5727
 Recall: 0.6446
```



Decision Tree

```
dtree = DecisionTreeClassifier()
dtree.fit(X_train, y_train)

y_pred_dtree = dtree.predict(X_test)

# Evaluate the model
accuracy_dtree = accuracy_score(y_test, y_pred_dtree)
precision_dtree = precision_score(y_test, y_pred_dtree,
average='weighted')
f1_dtree = f1_score(y_test, y_pred_dtree, average='weighted')
recall_dtree = recall_score(y_test, y_pred_dtree, average='weighted')

print('====== Decision Tree ======')
print(f" Accuracy: {accuracy_dtree:.4f}")
print(f" Precision: {precision_dtree:.4f}")
print(f" F1-score: {f1_dtree:.4f}")
print(f" Recall: {recall_dtree:.4f}")

# Confusion Matrix
cm_dtree = confusion_matrix(y_test, y_pred_dtree)
disp_dtree = ConfusionMatrixDisplay(confusion_matrix=cm_dtree,
display_labels=dtree.classes_)
disp_dtree.plot(cmap=plt.cm.Blues)
plt.title('Decision Tree Confusion Matrix')
plt.show()
```

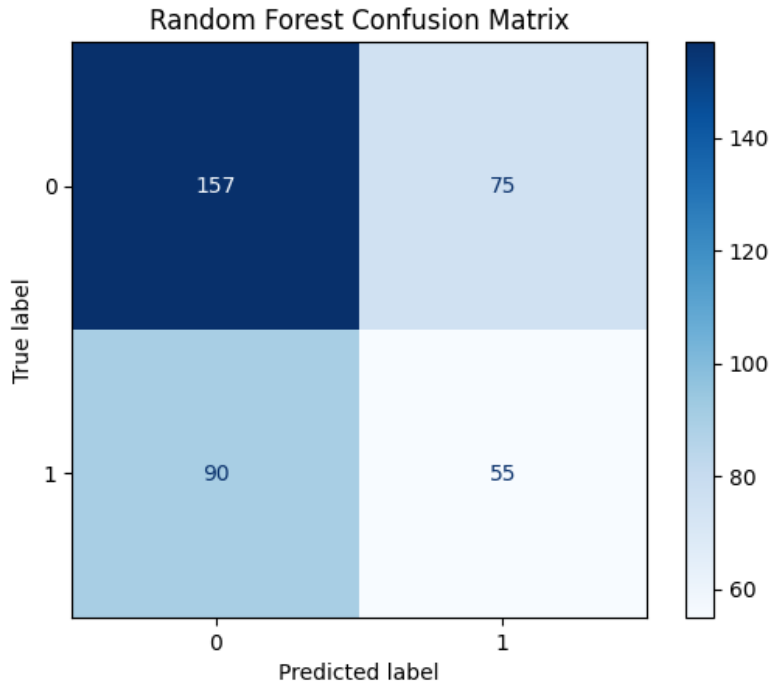
```
====== Decision Tree ======
 Accuracy: 0.5517
 Precision: 0.5430
 F1-score: 0.5465
 Recall: 0.5517
```



Random Forest

```
rf_model = RandomForestClassifier(n_estimators=100, random_state=0)
rf_model.fit(X_train, y_train)

y_pred_rf = rf_model.predict(X_test)

# Evaluate the model
accuracy_rf = accuracy_score(y_test, y_pred_rf)
precision_rf = precision_score(y_test, y_pred_rf, average='weighted')
f1_rf = f1_score(y_test, y_pred_rf, average='weighted')
recall_rf = recall_score(y_test, y_pred_rf, average='weighted')

print('====== Random Forest ======')
print(f" Accuracy: {accuracy_rf:.4f}")
print(f" Precision: {precision_rf:.4f}")
print(f" Recall: {recall_rf:.4f}")
print(f" F1-score: {f1_rf:.4f}")

# Confusion Matrix
cm_rf = confusion_matrix(y_test, y_pred_rf)
disp_rf = ConfusionMatrixDisplay(confusion_matrix=cm_rf,
display_labels=rf_model.classes_)
disp_rf.plot(cmap=plt.cm.Blues)
plt.title('Random Forest Confusion Matrix')
plt.show()
```

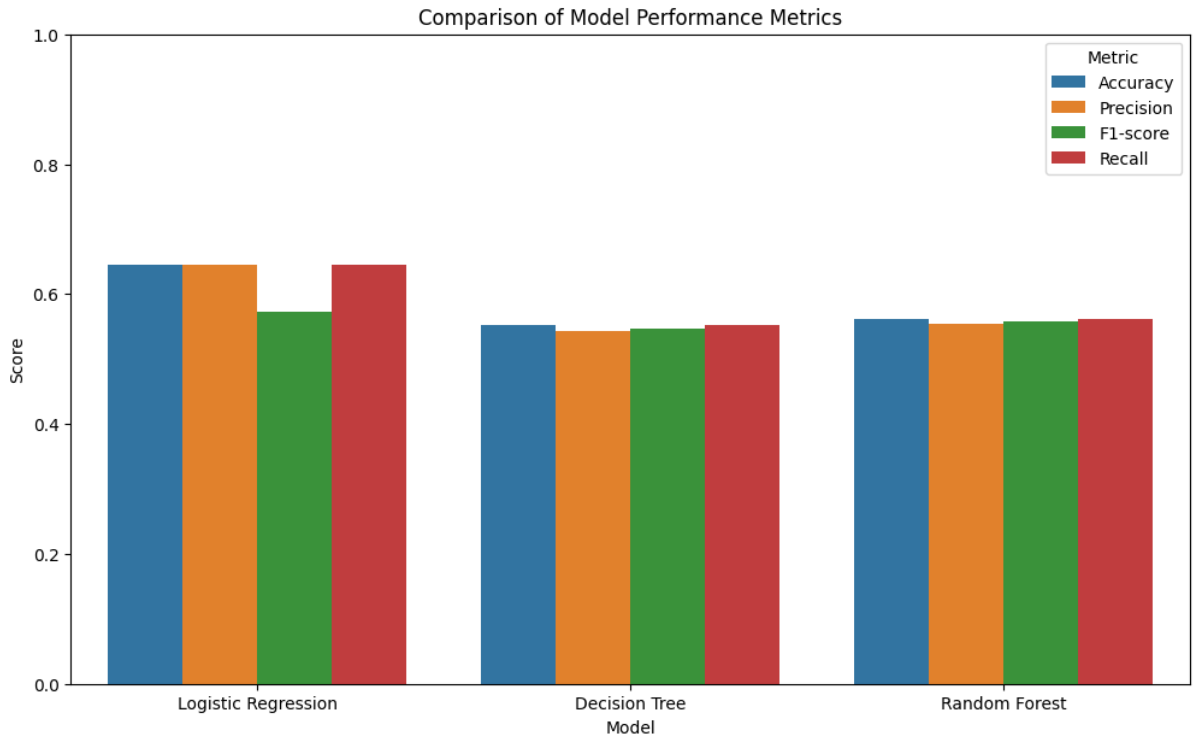
```
====== Random Forest ======
 Accuracy: 0.5623
 Precision: 0.5539
 Recall: 0.5623
 F1-score: 0.5573
```



Comparison between Models

```
results_df = pd.DataFrame({
    'Model': ['Logistic Regression', 'Decision Tree', 'Random Forest'],
    'Accuracy': [accuracy_logreg, accuracy_dtree, accuracy_rf],
    'Precision': [precision_logreg, precision_dtree, precision_rf],
    'F1-score': [f1_logreg, f1_dtree, f1_rf],
    'Recall': [recall_logreg, recall_dtree, recall_rf]
})

print(results_df.to_string(index=False))

results_melted = results_df.melt('Model', var_name='Metric',
value_name='Score')
plt.figure(figsize=(12, 7))
sns.barplot(x='Model', y='Score', hue='Metric', data=results_melted)
plt.title('Comparison of Model Performance Metrics')
plt.ylabel('Score')
plt.ylim(0, 1)
plt.show()
```

```
              Model  Accuracy  Precision  F1-score   Recall
Logistic Regression  0.644562   0.645860  0.572658 0.644562
      Decision Tree  0.551724   0.542977  0.546517 0.551724
      Random Forest  0.562334   0.553877  0.557251 0.562334
```



Model Performance and Evaluation

The performance of the three classification models – Logistic Regression, Decision Tree, and Random Forest was evaluated across three target variables:

1 care_options

Logistic Regression	Highest accuracy (52.52%) and precision (49.98%), but the F1-score (46.66%) indicated moderate performance in balancing precision and recall
Decision Tree	Performed moderately with an accuracy of 48.28% and an F1-score of 48.10%
Random Forest	Similar performance to the Decision Tree with an accuracy of 48.01% and an F1-score of 47.79%

Logistic Regression perform better than the other models for this target variable. It showed that a linear approach may better capture the relationship between the features and the availability of care options the employer provides.

2 treatment

Logistic Regression	Achieved an accuracy of 60.21% and an F1-score of 60.11%
Decision Tree	Lower accuracy (54.38%) and F1-score (54.45%), weaker predictive capability
Random Forest	Similar performance to the Decision Tree with an accuracy (60.21%) and F1-score (60.27%)

Random Forest performed well for predicting treatment-seeking behaviour. It showed that the model is suitable for binary classification tasks.

3 family_history

Logistic Regression	Highest accuracy (64.46%) and F1-score (57.27%), indicated moderate performance in balancing precision and recall
Decision Tree	Perform moderately with an accuracy of 55.17% and an F1-score of 54.65%
Random Forest	Slightly improved than the Decision Tree with an accuracy of 56.23% and an F1-score of 55.73%

Logistic Regression was the most effective model for predicting the influence of family history on mental health, likely due to the linear relationships in the data.

Conclusion

In this project, we used Python to study how mental health affects tech workers. We tried to predict three important things - how much care options provided by the employers affects people, if people get treatment, and if family history affect the workers' mental health as well. We used three different prediction method – Logistic Regression, Decision Tree, and Random Forest.

The analysis was pretty good at predicting if someone would get treatment (about 60% accurate) and if family history was important (64% right). The Logistic Regression method worked best for these. But predicting how much care options provided by the employers to the worker was harder. The prediction only got it right about half the time. This might be because people's feelings at work are more complicated to measure.

We learned that things like age, whether you work from home, and what your company offers for mental health help were the biggest clues for the prediction. Just like how older people might be more likely to get treatment, or people who work from home might feel less stressed.

One problem was that some answers were missing in our datasets, like when people didn't say how mental health affects their work. Also, some groups of people answered more than others, which made it harder for the model to learn. In the future, we could try better ways to fill in missing data or use stronger prediction methods like XGBoost.

Even though our predictions aren't perfect yet, they show we can use technology to help understand mental health at work. If we keep improving them, maybe one day they can help companies make better rules to keep their workers happy and healthy.

References

Breiman, L. (2001). Random Forests. *Machine Learning*, *45*(1), 5–32. https://doi.org/10.1023/a:1010933404324

GuyonIsabelle, & ElisseeffAndré. (2003). An introduction to variable and feature selection. *Journal of Machine Learning Research*. https://doi.org/10.5555/944919.944968

Hosmer, D. W., Lemeshow, S., & Sturdivant, R. X. (2013). Applied Logistic Regression. In *Wiley series in probability and statistics*. https://doi.org/10.1002/9781118548387

Mental Health in Tech survey. (2016, November 3). Kaggle. https://www.kaggle.com/datasets/osmi/mental-health-in-tech-survey

Safavian, S., & Landgrebe, D. (1991). A survey of decision tree classifier methodology. *IEEE Transactions on Systems Man and Cybernetics*, *21*(3), 660–674. https://doi.org/10.1109/21.97458

VanderPlas, J. (2023). *Python Data Science Handbook: Essential Tools for Working with Data*. O'Reilly Media.